Assignment 4

Name: Jakkarin Promsee
Student ID: 67070501009

0. Setup

In [44]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import sklearn

pd.set_option('display.max_columns', 100)
print("sklearn :", sklearn.__version__)
```

```
sklearn : 1.8.0
```

1. Load dataset

In [45]:
```
df = pd.read_csv('MBA.csv')

print("shape :", df.shape)
print("cols  :", list(df.columns))
df.head()
```

```
shape : (6194, 10)
cols  : ['application_id', 'gender', 'international', 'gpa', 'major', 'race', 'gmat', 'work_exp', 'work_industry',
'admission']
```

Out[45]:

	application_id	gender	international	gpa	major	race	gmat	work_exp	work_industry	admission
0	1	Female	False	3.30	Business	Asian	620.0	3.0	Financial Services	Admit
1	2	Male	False	3.28	Humanities	Black	680.0	5.0	Investment Management	NaN
2	3	Female	True	3.30	Business	NaN	710.0	5.0	Technology	Admit
3	4	Male	False	3.47	STEM	Black	690.0	6.0	Technology	NaN
4	5	Male	False	3.35	STEM	Hispanic	590.0	5.0	Consulting	NaN

In [46]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6194 entries, 0 to 6193
Data columns (total 10 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   application_id  6194 non-null   int64
 1   gender          6194 non-null   object
 2   international   6194 non-null   bool
 3   gpa             6194 non-null   float64
 4   major           6194 non-null   object
 5   race            4352 non-null   object
 6   gmat            6194 non-null   float64
 7   work_exp        6194 non-null   float64
 8   work_industry   6194 non-null   object
 9   admission       1000 non-null   object
dtypes: bool(1), float64(3), int64(1), object(5)
memory usage: 441.7+ KB
```

2. Target: admission -> admit 1/0

`admission` มี 3 ค่า Admit / Waitlist / NaN โดย **NaN = Deny**

In [47]:
```
df['admission'].value_counts()
```

Out[47]:
```
admission
Admit       900
Waitlist    100
Name: count, dtype: int64
```

In [48]:
```
df['admission'].value_counts(dropna=False)
```

Out[48]:
```
admission
NaN         5194
Admit        900
Waitlist     100
Name: count, dtype: int64
```

In [49]:
```
df['admit'] = (df['admission'] == 'Admit').astype(int)

# เช็คว่า map ถูก แถว NaN ต้องลงไปช่อง 0 ให้หมด
print(pd.crosstab(df['admission'].fillna('(NaN = Deny)'), df['admit']))
print("\nadmit rate = %.4f" % df['admit'].mean())
```

```
admit            0    1
admission
(NaN = Deny)  5194    0
Admit            0  900
Waitlist       100    0

admit rate = 0.1453
```

> positive class มีแค่ 14.5% -> imbalanced

3. Explore and clean data

In [50]:
```
# application_id เป็นเลขรัน ไม่ใช่ feature / admission ใช้ทำ target ไปแล้ว
df = df.drop(columns=['application_id', 'admission'])
```

In [51]:
```
df.shape
```

Out[51]:
```
(6194, 9)
```

In [52]:
```
df.isna().sum()
```

Out[52]:
```
gender              0
international       0
gpa                 0
major               0
race             1842
gmat                0
work_exp            0
work_industry       0
admit               0
dtype: int64
```

In [53]:
```
df.select_dtypes('number').describe()
```

Out[53]:

	gpa	gmat	work_exp	admit
count	6194.000000	6194.000000	6194.000000	6194.000000
mean	3.250714	651.092993	5.016952	0.145302
std	0.151541	49.294883	1.032432	0.352433
min	2.650000	570.000000	1.000000	0.000000
25%	3.150000	610.000000	4.000000	0.000000
50%	3.250000	650.000000	5.000000	0.000000
75%	3.350000	680.000000	6.000000	0.000000
max	3.770000	780.000000	9.000000	1.000000

In [54]:
```
df.select_dtypes(['object', 'bool']).describe()
```

Out[54]:

	gender	international	major	race	work_industry
count	6194	6194	6194	4352	6194
unique	2	2	3	5	14
top	Male	False	Humanities	White	Consulting
freq	3943	4352	2481	1456	1619

In [55]:
```
for c in df.select_dtypes('object').columns:
    print(df[c].value_counts(dropna=False), "\n")
```

```
gender
Male      3943
Female    2251
Name: count, dtype: int64

major
Humanities    2481
STEM          1875
Business      1838
Name: count, dtype: int64

race
NaN         1842
White       1456
Asian       1147
Black        916
Hispanic     596
Other        237
Name: count, dtype: int64

work_industry
Consulting               1619
PE/VC                     907
Technology                716
Nonprofit/Gov             651
Investment Banking        580
Financial Services        451
Other                     421
Health Care               334
Investment Management     166
CPG                       114
Real Estate               111
Media/Entertainment        59
Retail                     33
Energy                     32
Name: count, dtype: int64
```

4. race ที่ว่างอยู่ 1842 แถว

เช็คก่อนว่าหายแบบสุ่มหรือหายมีแพทเทิร์น ค่อยตัดสินใจว่าจะ drop หรือ fill

In [56]:

```
print(pd.crosstab(df['race'].isna(), df['international']))
```

```
international  False  True
race
False           4352     0
True               0  1842
```

In [57]:

```
df['race'] = df['race'].fillna('International')

# เติมแล้ว race_International จะซ้ำกับ international ทุกแถว เก็บทั้งคู่ importance โดนหารกัน
print("duplicate :", (df['race'] == 'International').equals(df['international']))
df = df.drop(columns='international')

print(df['race'].value_counts())
```

```
duplicate : True
race
International    1842
White            1456
Asian            1147
Black             916
Hispanic          596
Other             237
Name: count, dtype: int64
```

5. Encoding

In [58]:

```
df_num = df.select_dtypes('number').drop(columns='admit')
df_cat = df.select_dtypes('object')
```

```
print("numeric     :", list(df_num.columns))
print("categorical :", list(df_cat.columns))
```

```
numeric     : ['gpa', 'gmat', 'work_exp']
categorical : ['gender', 'major', 'race', 'work_industry']
```

In [59]:

```
df_cat_bin = pd.get_dummies(df_cat, drop_first=True)

print("shape :", df_cat_bin.shape)
df_cat_bin.head()
```

```
shape : (6194, 21)
```

Out[59]:

	gender_Male	major_Humanities	major_STEM	race_Black	race_Hispanic	race_International	race_Other	race_White	work
0	False	False	False	False	False	False	False	False	
1	True	True	False	True	False	False	False	False	
2	False	False	False	False	False	True	False	False	
3	True	False	True	True	False	False	False	False	
4	True	False	True	False	True	False	False	False	

In [60]:

```
df_prep = pd.concat([df_num, df_cat_bin], axis=1)

print("shape :", df_prep.shape)
df_prep.head()
```

```
shape : (6194, 24)
```

Out[60]:

	gpa	gmat	work_exp	gender_Male	major_Humanities	major_STEM	race_Black	race_Hispanic	race_International	race_C
0	3.30	620.0	3.0	False	False	False	False	False	False	
1	3.28	680.0	5.0	True	True	False	True	False	False	
2	3.30	710.0	5.0	False	False	False	False	False	True	
3	3.47	690.0	6.0	True	False	True	True	False	False	
4	3.35	590.0	5.0	True	False	True	False	True	False	

6. Holdout

In [61]:

```
from sklearn.model_selection import train_test_split

X = df_prep
y = df['admit']

# stratify กัน admit ที่มีน้อยอยู่แล้วไปกระจุกฝั่งเดียว
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=777, stratify=y)

print("train :", X_train.shape, "  test :", X_test.shape)
print("train admit rate :", round(y_train.mean(), 4))
print("test  admit rate :", round(y_test.mean(), 4))
```

```
train : (4335, 24)   test : (1859, 24)
train admit rate : 0.1453
test  admit rate : 0.1452
```

7. tools

In [62]:

```
from sklearn.metrics import (classification_report, confusion_matrix, roc_curve, auc,
                             accuracy_score, precision_score, recall_score, f1_score)

scores = {}
curves = {}

def evaluate(name, model):
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    print(confusion_matrix(y_test, y_pred), "\n")
    print(classification_report(y_test, y_pred, digits=3))

    fpr, tpr, _ = roc_curve(y_test, y_prob)
    curves[name] = (fpr, tpr)
    scores[name] = {
        'accuracy':  accuracy_score(y_test, y_pred),
        'precision': precision_score(y_test, y_pred),
        'recall':    recall_score(y_test, y_pred),
        'f1':        f1_score(y_test, y_pred),
        'AUC':       auc(fpr, tpr),
    }
    print("AUC = %.4f" % scores[name]['AUC'])

def top_features(model, n=15):
    return (pd.DataFrame(dict(Feature=model.feature_names_in_,
                              Value=model.feature_importances_))
              .sort_values(by='Value', ascending=False)
              .head(n))
```

8. Decision tree

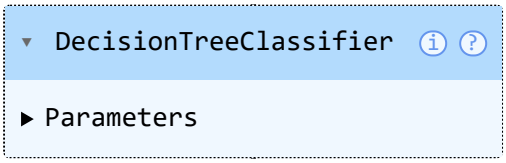
In [ ]:

```
%%time
from sklearn.tree import DecisionTreeClassifier

tree = DecisionTreeClassifier(max_depth=8, min_samples_leaf=20,
                              class_weight='balanced',
                              random_state=777)
tree.fit(X_train, y_train)
```

```
CPU times: total: 31.2 ms
Wall time: 10 ms
```

Out[ ]:



In [64]:

```
evaluate('Decision tree', tree)
```

```
[[1155  434]
 [  49  221]]

              precision    recall  f1-score   support

           0      0.959     0.727     0.827      1589
           1      0.337     0.819     0.478       270

    accuracy                          0.740      1859
   macro avg      0.648     0.773     0.652      1859
weighted avg      0.869     0.740     0.776      1859

AUC = 0.8366
```

In [65]:

```
top_features(tree)
```

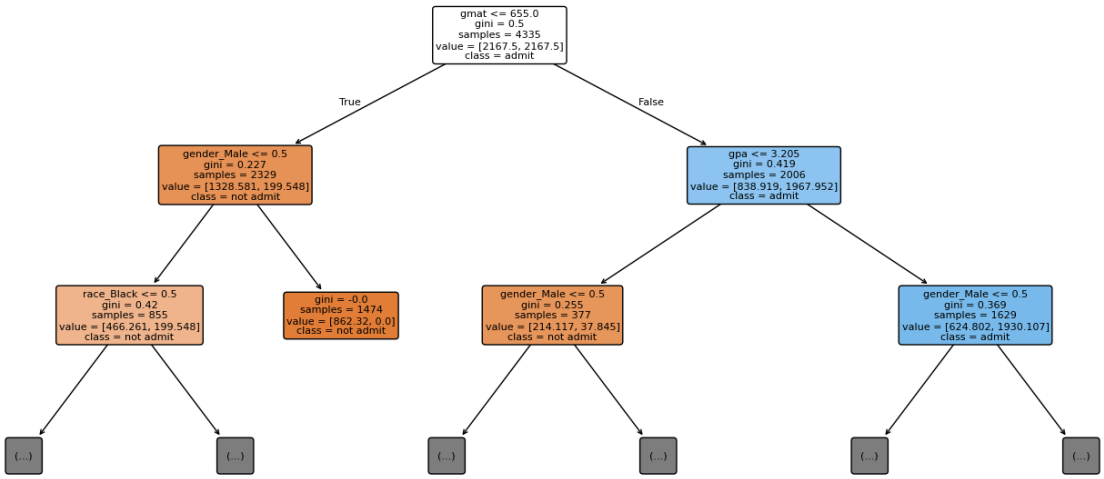
Out[65]:

	Feature	Value
1	gmat	0.633516
0	gpa	0.187049
3	gender_Male	0.094738
6	race_Black	0.025605
7	race_Hispanic	0.015501
8	race_International	0.011394
2	work_exp	0.010300
11	work_industry_Consulting	0.008046
4	major_Humanities	0.007133
9	race_Other	0.004756
20	work_industry_PE/VC	0.001153
19	work_industry_Other	0.000809
5	major_STEM	0.000000
13	work_industry_Financial Services	0.000000
12	work_industry_Energy	0.000000

In [66]:

```
from sklearn.tree import plot_tree

# โชว์แค่ 2 ชั้นพอ ทั้งต้น (depth 8) พิมพ์ออกมาอ่านไม่ออก
plt.figure(figsize=(18, 8))
plot_tree(tree, max_depth=2, feature_names=X_train.columns,
          class_names=['not admit', 'admit'], filled=True, rounded=True, fontsize=8)
plt.show()
```



9. Random forest

In [ ]:

```
%%time
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=300, min_samples_leaf=5,
                            class_weight='balanced_subsample',
                            oob_score=True,
                            random_state=777, n_jobs=-1)
rf.fit(X_train, y_train)

print("oob score :", rf.oob_score_)
```

```
oob score : 0.7670126874279123
CPU times: total: 1.17 s
Wall time: 775 ms
```

In [68]:

```
evaluate('Random forest', rf)
```

```
[[1206  383]
 [  53  217]]

              precision    recall  f1-score   support

           0      0.958     0.759     0.847      1589
           1      0.362     0.804     0.499       270

    accuracy                          0.765      1859
   macro avg      0.660     0.781     0.673      1859
weighted avg      0.871     0.765     0.796      1859

AUC = 0.8507
```

In [69]:

```
top_features(rf)
```

Out[69]:

	Feature	Value
1	gmat	0.443050
0	gpa	0.302619
3	gender_Male	0.068402
2	work_exp	0.036570
6	race_Black	0.017507
4	major_Humanities	0.014453
8	race_International	0.014048
5	major_STEM	0.013842
11	work_industry_Consulting	0.011722
10	race_White	0.011062
7	race_Hispanic	0.010023
20	work_industry_PE/VC	0.007800
13	work_industry_Financial Services	0.007763
18	work_industry_Nonprofit/Gov	0.007064
15	work_industry_Investment Banking	0.006427

10. Gradient boosting

In [70]:

```
%%time
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.utils.class_weight import compute_sample_weight

gb = GradientBoostingClassifier(n_estimators=300, learning_rate=0.05, max_depth=3,
                                random_state=777)
gb.fit(X_train, y_train, sample_weight=compute_sample_weight('balanced', y_train))
```

```
CPU times: total: 875 ms
Wall time: 862 ms
```

Out[70]:

GradientBoostingClassifier

▶ Parameters

In [71]:

```
evaluate('Gradient boosting', gb)
```

```
[[1143  446]
 [  36  234]]

              precision    recall  f1-score   support

           0      0.969     0.719     0.826      1589
           1      0.344     0.867     0.493       270

    accuracy                          0.741      1859
   macro avg      0.657     0.793     0.659      1859
weighted avg      0.879     0.741     0.777      1859

AUC = 0.8451
```

In [72]:

```
top_features(gb)
```

Out[72]:

	Feature	Value
1	gmat	0.601661
0	gpa	0.198863
3	gender_Male	0.101911
6	race_Black	0.025674
2	work_exp	0.014942
7	race_Hispanic	0.012602
9	race_Other	0.006216
8	race_International	0.006170
13	work_industry_Financial Services	0.004996
4	major_Humanities	0.003906
16	work_industry_Investment Management	0.003195
5	major_STEM	0.002950
19	work_industry_Other	0.002770
18	work_industry_Nonprofit/Gov	0.002546
20	work_industry_PE/VC	0.002242

11. Compare

In [73]:

```
compare = pd.DataFrame(scores).T
print(compare.round(4).to_string())
```

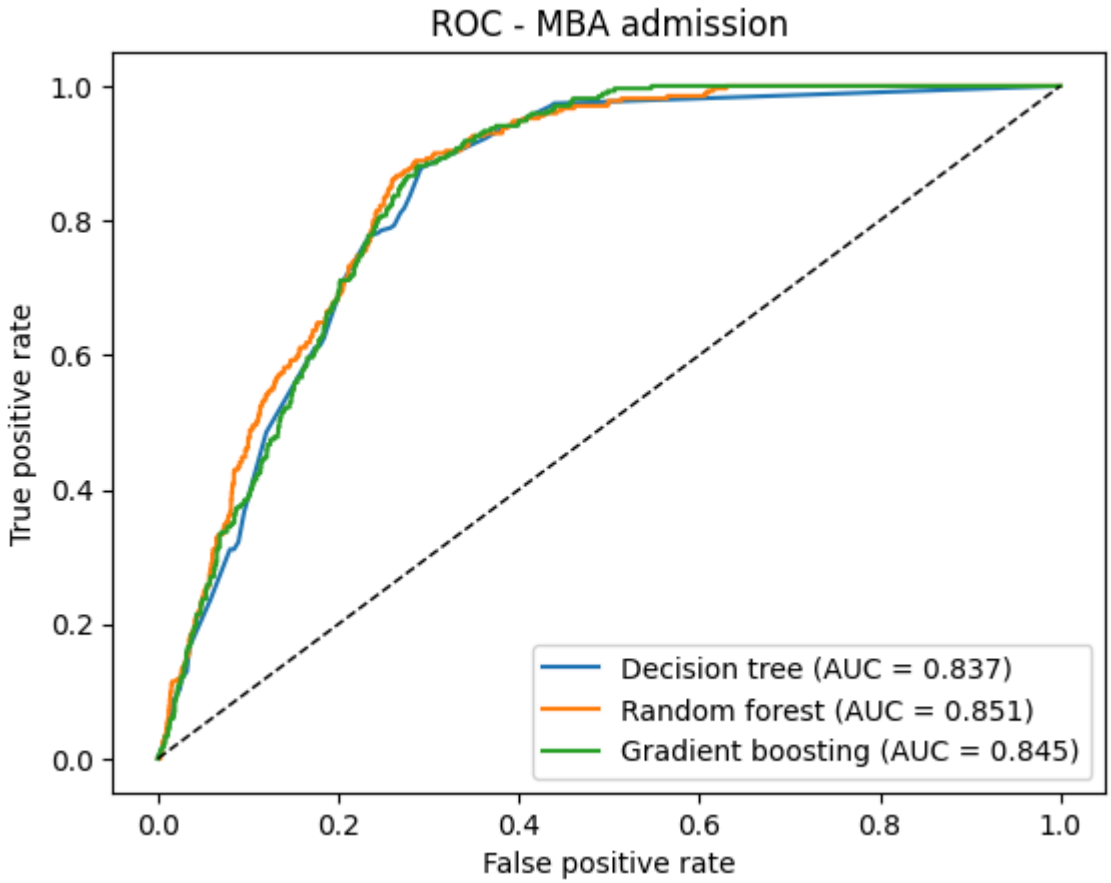
```
                  accuracy  precision  recall      f1     AUC
Decision tree       0.7402     0.3374  0.8185  0.4778  0.8366
Random forest       0.7655     0.3617  0.8037  0.4989  0.8507
Gradient boosting   0.7407     0.3441  0.8667  0.4926  0.8451
```

In [ ]:

```
ax = plt.subplot(111)

for name, (fpr, tpr) in curves.items():
    ax.plot(fpr, tpr, label="%s (AUC = %.3f)" % (name, scores[name]['AUC']))

ax.plot([0, 1], [0, 1], 'k--', lw=1)
plt.title('ROC - MBA admission')
plt.xlabel('False positive rate')
plt.ylabel('True positive rate')
plt.legend()
plt.show()
```



12. Summary

- โมเดลที่ AUC ดีที่สุดคือ Random forest (AUC = 0.851) รองลงมาคือ Gradient boosting และ Decision tree
- ตัวแปรที่ตัดสินผล admit มากที่สุดคือ gmat กับ gpa ที่ตรงกันจากทั้ง 3 โมเดล
- accuracy ทุกโมเดลอยู่ประมาณ 0.75 แต่ precision ของฝั่ง admit ต่ำกว่ามาก เพราะว่าเราใช้ class_weight='balanced' (ข้อมูลมัน imbalance) ซึ่งเราแลก precision ไปเอา recall ที่สำคัญกว่า
- เพราะในมุมมองของคนคัดใบสมัคร recall สำคัญกว่า precision เนื่องจากไม่ต้องการ False Negative ที่ คนที่ได้ admit จริงแต่โมเดลบอกไม่ผ่าน